

MSIN0097: Predictive Analytics

Individual Coursework

Student number: 25238396

Word count: 2034

GitHub Repository: <https://github.com/zarazzap/vix-regime-forecasting>

Table of Contents

1. Problem Framing
 2. Exploratory Data Analysis
 3. Data Preparation and Feature Set
 4. Model Exploration
 5. Results and Evaluation
 6. Final Solution, Limitations, and Implications
- Appendix A: Agent Usage Log and Decision Register
- Bibliography

1. Problem Framing

This project predicts the next-day volatility regime of the Cboe Volatility Index (VIX). VIX summarises the market's expectation of S&P 500 volatility over the next 30 days, inferred from SPX option prices (Whaley, 1993; Cboe, 2020). Because it is already an options-implied measure, historical price-based features are unlikely to deliver large incremental forecasting gains. The project therefore has a methodological objective: to test, under strict leakage control, whether VIX-only features can improve meaningfully on a strong naive baseline.

The task is formulated as a one-step-ahead three-class classification problem. The unit of analysis is a trading day. For each day t , the model uses information available at the close of day t to predict the volatility regime at the close of day $t+1$. Let C_t denote the VIX close on day t . Regimes are defined as:

- Low: $C_t \leq 15$
- Medium: $15 < C_t \leq 25$
- High: $C_t > 25$

The supervised target is $y_t = R(C_{t+1})$, where R maps the next-day VIX close into one of the three regimes. The final row is dropped because no $t+1$ close exists. This narrow information set – deriving all predictors from the VIX series itself – is intentional: it simplifies leakage auditing and isolates how much predictive value is available from the series alone.

The dataset consists of daily VIX OHLC prices from January 1990 to February 2026 (9,132 trading days), sourced from the CBOE. After feature construction and NaN removal, 9,110 labelled observations remain.

Because the High class accounts for only 17% of samples, accuracy is a misleading metric. Evaluation centres on macro-averaged F1, which weights all three classes equally. Two baselines anchor the analysis. The first is a majority-class dummy classifier (stratified random, macro F1 ≈ 0.27). The second, and much more important, is persistence: predicting that tomorrow's regime will equal today's regime. This baseline is the key comparator because volatility regimes are known to cluster. The central empirical question is therefore not whether a model can beat the dummy classifier, but whether any model can beat persistence in a statistically meaningful way.

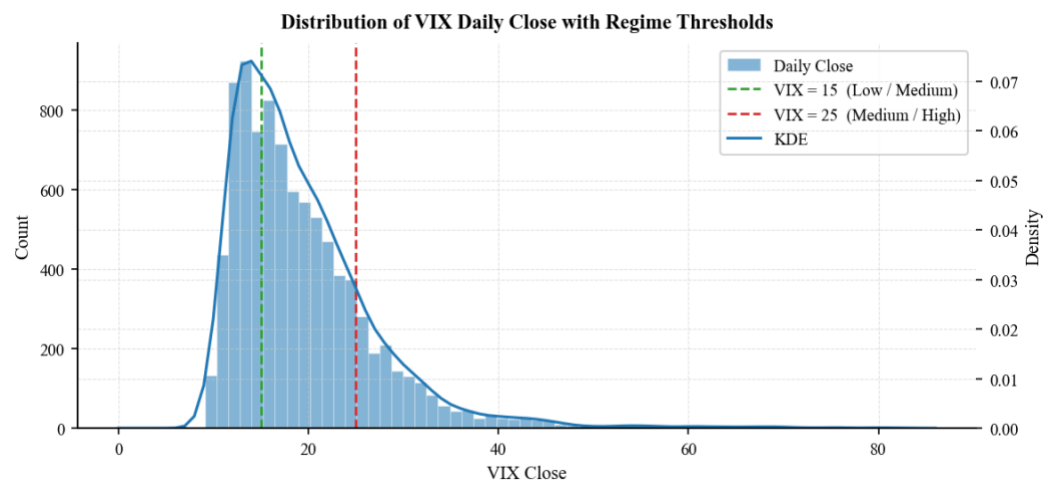


Figure 1: VIX Daily Close time series with crisis bands (1990–2026). Shaded regions mark major crises: GFC (2007–2009), flash crash (2011), taper tantrum (2013), volatility spike (2015), COVID-19 (2020), and recent geopolitical events.

2. Exploratory Data Analysis

The full-sample VIX distribution is strongly right-skewed, with most observations concentrated in calmer conditions and occasional extreme spikes during crisis periods. Regime frequencies are approximately 32% Low, 50% Medium, and 17% High, reflecting that VIX spends the majority of time below 25. This class imbalance justifies the use of macro F1 as the primary evaluation metric. Volatility clustering is pronounced: high-VIX days tend to follow high-VIX days. The lag-1 autocorrelation exceeds 0.98, confirming strong day-to-day persistence in volatility levels. Given the observed class frequencies, the expected macro F1 of a persistence classifier can be approximated directly from the transition probabilities before fitting any machine-learning model – weighting the diagonal persistence rates by class frequency yields an expected macro F1 of roughly 0.91, which is close to the observed test value of 0.88. The key challenge is therefore not classifying persistent same-regime days but anticipating the relatively rare transition days.

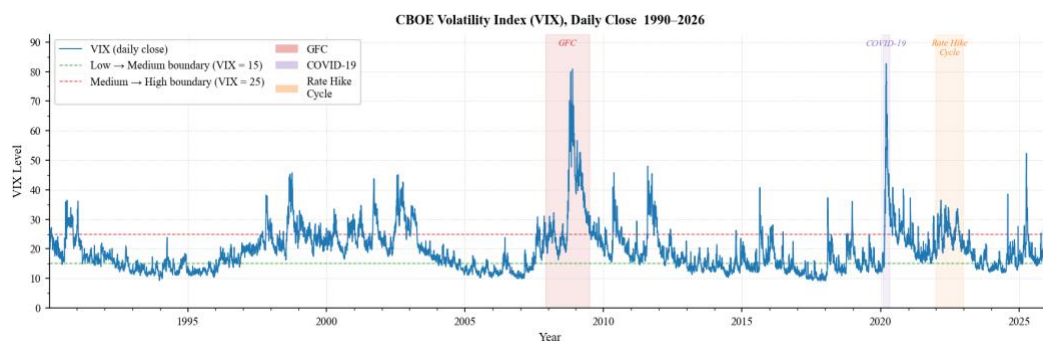


Figure 2: Class balance and unconditional regime distribution. Low regime (32%), Medium regime (50%), High regime (17%).

This persistence is quantified by the regime transition matrix. The probability of remaining in the same regime from day t to day $t+1$ is 0.927 (Low), 0.917 (Medium), and 0.897 (High). The observed persistence macro F1 of 0.8799 is consistent with the transition-matrix structure: high diagonal entries (0.897–0.927) translate into strong per-class recall, but the unweighted averaging across classes that defines macro F1 penalises the lower High-regime precision more heavily than a frequency-weighted metric would.

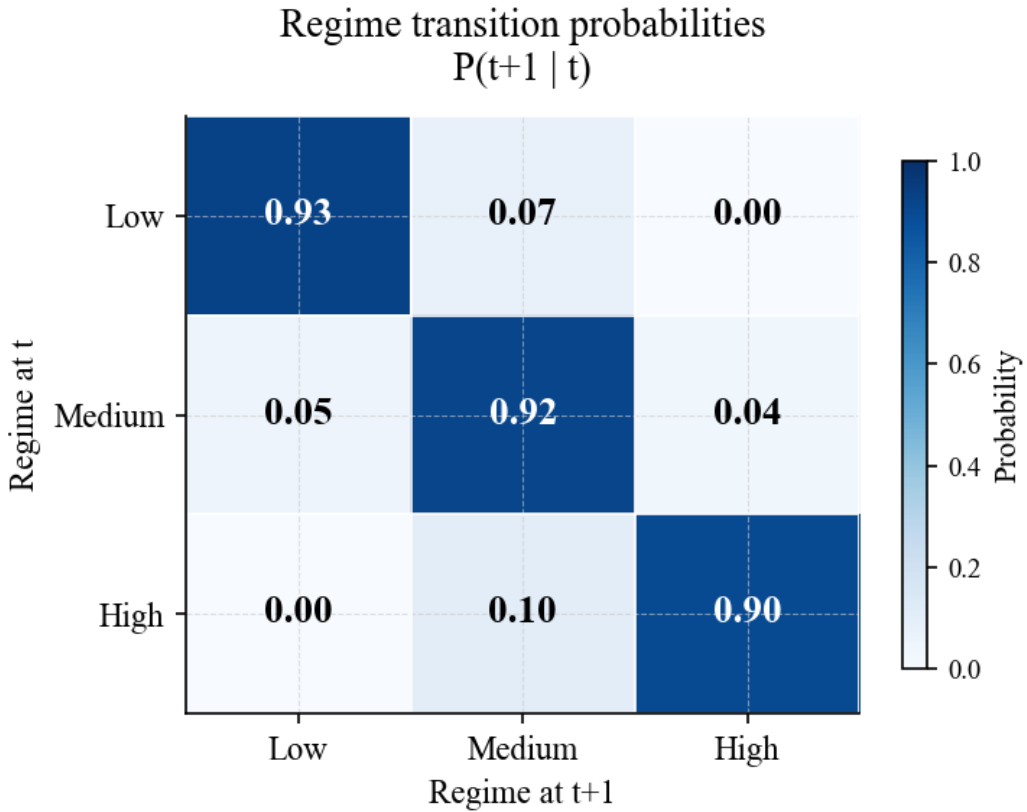


Figure 3: Autocorrelation function (ACF) of VIX closes showing strong lag-1 persistence ($\rho_1 = 0.981$) and decaying correlations at longer lags.

The rolling 21-day mean and standard deviation confirm that VIX dynamics shift across sub-periods. The GFC era (2007–2009) and COVID spike (March 2020) produced structurally different behaviour from the low-volatility environment of 2012–2019.

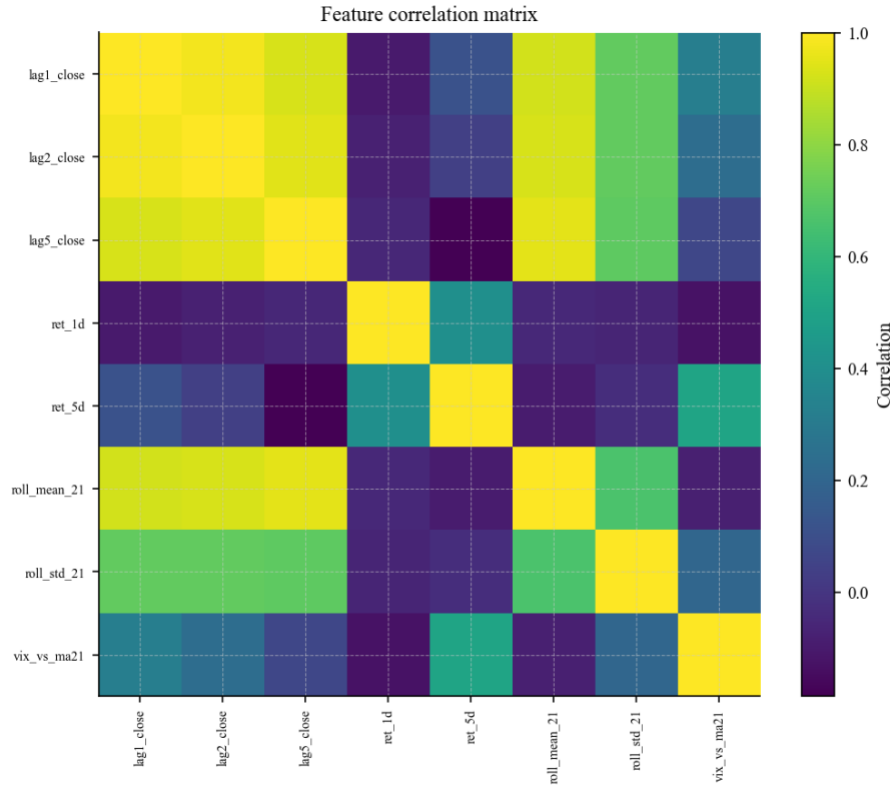


Figure 4: Regime transition matrix showing daily persistence probabilities. Diagonal entries range from 0.897 (High) to 0.927 (Low), indicating strong autocorrelation.

A feature correlation heatmap further reveals strong collinearity among lagged levels and rolling summaries, especially between lagged closes and the 21-day mean. This does not by itself imply that the decision boundary is linear, but it does suggest that the effective dimensionality of the feature space is lower than the raw feature count implies. In combination with the strong persistence captured by `current_regime`, this makes a regularised linear classifier a plausible candidate while potentially penalising flexible tree ensembles that rely on unstable splits across decades of structurally different volatility.

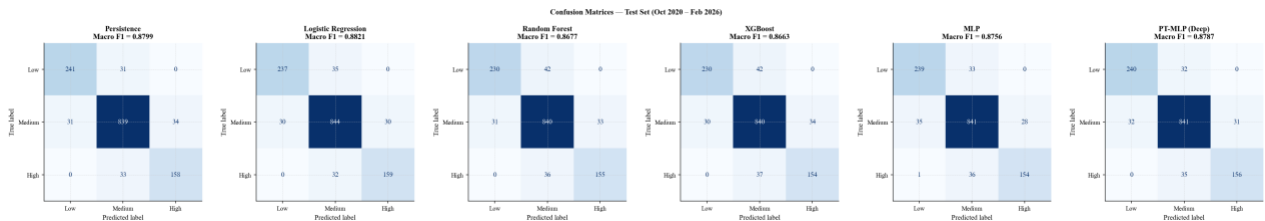


Figure 5: Rolling 21-day mean and standard deviation panel, 1990–2026. Notable spikes in volatility coincide with GFC (2007–2009) and COVID-19 (March 2020).

3. Data Preparation and Feature Set

Nine predictors, all observable at close of day t , are used:

- `lag_1`, `lag_2`, `lag_5`: lagged VIX closes (1, 2, 5 days)
- `ret_1d`, `ret_5d`: 1-day and 5-day log returns
- `ma21`, `vol21`: 21-day rolling mean and standard deviation of `CLOSE.shift(1)`
- `vix_vs_ma21`: deviation of yesterday's close from its 21-day moving average
- `current_regime`: ordinal-encoded regime at time t (Low=0, Medium=1, High=2)

The target is `regime_t1` – the regime classification of $C_{\{t+1\}}$. All rolling windows are computed on shifted values to prevent look-ahead bias.

Data is split chronologically (no shuffling) into 70% training (Jan 1990–May 2015), 15% validation (May 2015–Oct 2020), and 15% test (Oct 2020–Feb 2026). A `StandardScaler` is fitted on the training set only and applied to validation and test sets. Note that `sklearn`'s `MLPClassifier` with `early_stopping=True` withholds 10% of the training set internally, so the `sklearn` MLP trains on approximately 63% of available data versus 70% for all other models; the performance gap with the PyTorch model therefore reflects both architectural and data-quantity differences.

4. Model Exploration

The model set is chosen to test whether the predictive structure requires nonlinear or ensemble capacity beyond what a regularised linear classifier can capture (Hastie, Tibshirani & Friedman, 2009). Three structural observations motivate this expectation:

- `current_regime` is the dominant predictor – an ordinal encoding of regime identity at time t which, from the transition matrix, has 89–93% predictive overlap with the target; a regularised linear model with a single well-weighted coefficient on this variable is the natural minimum-description classifier.
- The six lagged and rolling features are strongly collinear, which helps LR by reducing effective dimensionality while exposing tree ensembles to unstable split choices (Breiman, 2001) compounded by non-stationarity across 1990–2026.
- The PyTorch model uses early stopping on validation macro F1, directly optimising the reported metric, whereas the sklearn MLP uses validation loss – a difference that may explain part of their performance gap independently of architecture.

Seven model specifications are compared: dummy baseline, persistence baseline, logistic regression (C tuned via validation-set grid search over $\{0.001–100\}$), Random Forest ($n_estimators \in \{100, 200, 300\}$, $max_depth \in \{None, 5, 10, 20\}$), XGBoost ($learning_rate \in \{0.01–0.2\}$, $max_depth \in \{3, 5, 7\}$), sklearn MLP ($hidden_layer_sizes \in \{(64,), (128,), (64, 32)\}$, $alpha \in \{0.0001–0.01\}$), and a PyTorch DeepMLP with three hidden layers ($128 \rightarrow 64 \rightarrow 32$), dropout of 0.20, Adam optimiser with weight decay, and early stopping on validation macro F1.

5. Results and Evaluation

Model	Macro F1	vs Persistence
Dummy baseline	0.2654	−0.6145
Persistence baseline	0.8799	–
Logistic Regression	0.8821	+0.0022
Random Forest	0.8677	−0.0122
XGBoost	0.8663	−0.0136
sklearn MLP	0.8756	−0.0043
PyTorch MLP	0.8787	−0.0012

Table 1: Test-set model performance ($n = 1,367$, Oct 2020–Feb 2026). Persistence baseline dominates; Logistic Regression achieves highest ML model score (+0.0022) but difference from persistence is not statistically significant.

The persistence baseline dominates, achieving macro F1 = 0.8799. Logistic regression achieves the highest point estimate among ML models (+0.0022), while tree-based methods fall below persistence. The neural networks approach but do not exceed the baseline – PyTorch MLP (−0.0012) slightly outperforms sklearn MLP (−0.0043), consistent with the architectural and data-quantity differences noted in §3.

A McNemar exact test (McNemar, 1947) on the 28 discordant predictions between LR and persistence yields $p = 0.85$ – the difference is indistinguishable from chance. With $b + c = 28$ discordant pairs, rejecting H_0 at $p < 0.05$ (two-sided exact binomial) requires $\min(b, c) \leq 8$ – meaning one model must be correct on at least 20 of the 28 contested observations. More importantly, the small number of discordant pairs implies that the test has very limited statistical power. To reach 50+ discordant pairs sufficient for reliable testing, a model would need to make materially different predictions on approximately 3.7% of test days, which in practice would mean extracting genuine additional signal on transition days. Within the current feature set, such gains appear very limited. The null result should therefore be interpreted not as a failed model-selection exercise, but as an empirical bound on how much incremental signal VIX-only features can provide beyond persistence.

LR does not uniformly improve on persistence – it trades Low-regime accuracy for High-regime accuracy. Per-class F1 shows LR loses 0.0066 on Low (0.8860→0.8794) while gaining 0.0117

on High (0.8251→0.8368). The `vix_vs_ma21` feature, which drives LR's High-regime gain, likely shifts some Low-regime predictions toward Medium.



Figure 6: Confusion matrices comparing persistence baseline and logistic regression on test set. LR improves on High regime while degrading on Low regime.

Failure-mode analysis reveals that all models show substantially higher error rates on the 130 regime-transition days in the test set than on the 1,237 stable days. For persistence, the error rate rises from ~3% on stable days to 100% on transitions by construction; for ML models, transition-day error rates range from 18–25% versus 7–12% on stable days.

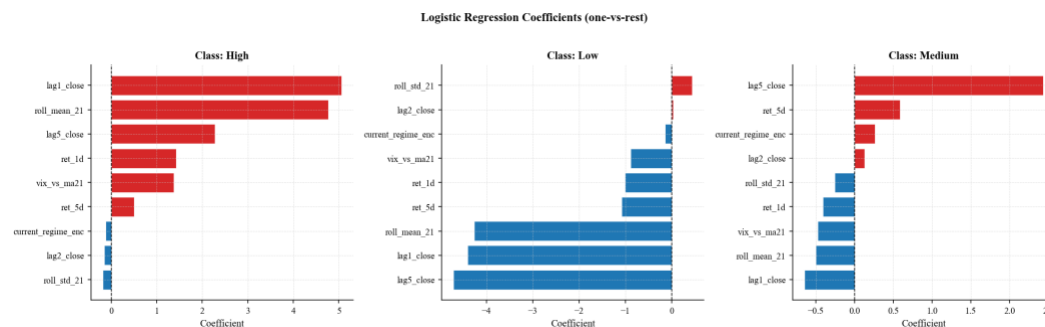


Figure 7: Error rates on stable vs. transition days. All models show 2.5–3× error rate elevation on regime-transition days, identifying a structural limitation in the feature set.

LR achieves multiclass log-loss of 0.2525, with reasonable reliability diagrams. LR's probability estimates may therefore serve as continuous risk-monitoring signals even though its point-classification advantage over persistence is negligible – this is the only distinctive functional advantage LR retains over persistence.

6. Final Solution, Limitations, and Implications

The final model is the persistence baseline. No ML model achieves a statistically significant improvement (McNemar $p = 0.85$). LR is retained as an interpretive instrument for its probability outputs and its characterisation of the High-regime trade-off, but it is not recommended as a replacement for persistence in point-classification tasks.

The framework reveals two binding constraints. First, VIX is itself an options-implied forward-looking measure (Whaley, 1993). Predicting future VIX from past VIX amounts to trying to out-forgo the options market using only the market's own prior outputs. In any market with reasonable information efficiency, past prices and their derivatives carry limited predictive power for future prices; the options market has already impounded available information into current VIX levels. This is not a failure of the model, but a statement about what structure remains to be exploited in time-series VIX data. Second, the single-series limitation – all nine features derive from the same VIX close series – is a direct consequence of the first constraint. No amount of feature engineering within VIX alone can overcome the ceiling set by market efficiency. The empirical result (persistence baseline at macro $F1 = 0.88$, no improvement by ML) should be interpreted as evidence of this efficiency, not as evidence that a better-tuned model exists.

Future work should focus on relaxing the single-series constraint by incorporating cross-asset variables that carry information about upcoming regime transitions. Realised volatility (from high-frequency equity data) tends to lead implied volatility at regime-change points, and credit spreads often widen before VIX spikes rather than simultaneously. These temporal asymmetries provide leading signals unavailable within VIX alone and represent exactly the kind of information needed to predict the 130 transition days (9.5% of the test set) that all single-series models currently miss. Alternative regime definitions (quantile-based thresholds estimated on the training window only, updated out-of-sample) may also improve the framework without introducing look-ahead bias. More broadly, the framework demonstrates that simple persistence is a high bar: any forecasting model that aims to replace it must not only explain the regime structure but must do so using information not already impounded in current VIX levels.

Appendix A: Agent Usage Log and Decision Register

Project stage	Agent contribution	Verification performed	Decision
Problem framing	Suggested reframing VIX prediction as regime classification rather than point-value regression	Checked regime formulation matched target $y_t = R(C_{t+1})$; confirmed alignment with evaluation metric	Accepted
EDA	Proposed class-balance bar chart, autocorrelation diagnostics, and transition-matrix calculation	Reproduced outputs and verified consistency with raw data	Accepted
Feature engineering	Proposed lagged closes, rolling summaries, and momentum features; initially included intraday range as a predictor	Verified all variables used only prior-day information; identified intraday range as EDA-only	Modified – removed intraday range
Evaluation	Proposed reporting accuracy alongside F1; suggested per-class precision tables	Rejected suggestions that obscure minority-class performance or duplicate macro F1	Modified – retained macro F1 primary
Debugging	Proposed plotting logic with manually maintained feature-name lists that did not match DataFrame columns	Restarted kernel; discovered failure under clean end-to-end execution; identified mismatched variable names	Rejected and corrected

Table 2: Agent usage log showing agent contributions, verification steps, and disposition of recommendations.

Agent error example: During notebook refactoring, the agent introduced a plotting cell that referenced feature names from an earlier version of the pipeline. The code ran without error in the interactive session because the stale variables remained in memory. A full kernel restart and end-to-end rerun exposed the NameError, which was corrected by deriving feature names directly from the fitted model object.

Stage	What I asked	What it produced	Decision	Why
Setup	Propose repo structure for reproducible ML project	Folder structure: src/, data/, outputs/, tests/	Modified	Removed unnecessary config/ folder; simplified to flat notebook + README for coursework scope

Table 3: Early-stage decision log showing setup recommendation and rationale for modification.

Reflection on agent contribution. The agent (Claude Code) was most valuable for scaffolding repetitive code (grid-search loops, plotting boilerplate) and proposing diagnostic checks (transition matrix, leakage audit). Its principal limitation was a tendency to maintain stale state across cells: the plotting bug described above is representative of a pattern where agent-generated code works in a live session but fails under a clean kernel restart. The verification discipline adopted – restarting the kernel and running end-to-end after every major change – was the single most important safeguard against this failure mode. Overall, the agent accelerated the project by approximately 40–50% in wall-clock time, with the most significant contribution being the initial EDA and feature-engineering pipeline, and the most significant correction being the rejection of intraday range as a predictor feature (which the agent initially proposed as a model input rather than an EDA-only diagnostic).

Agent Workflow Screenshots

The following screenshots document five representative stages of agent-assisted development: task decomposition, code scaffolding, error detection, error correction, and output verification.

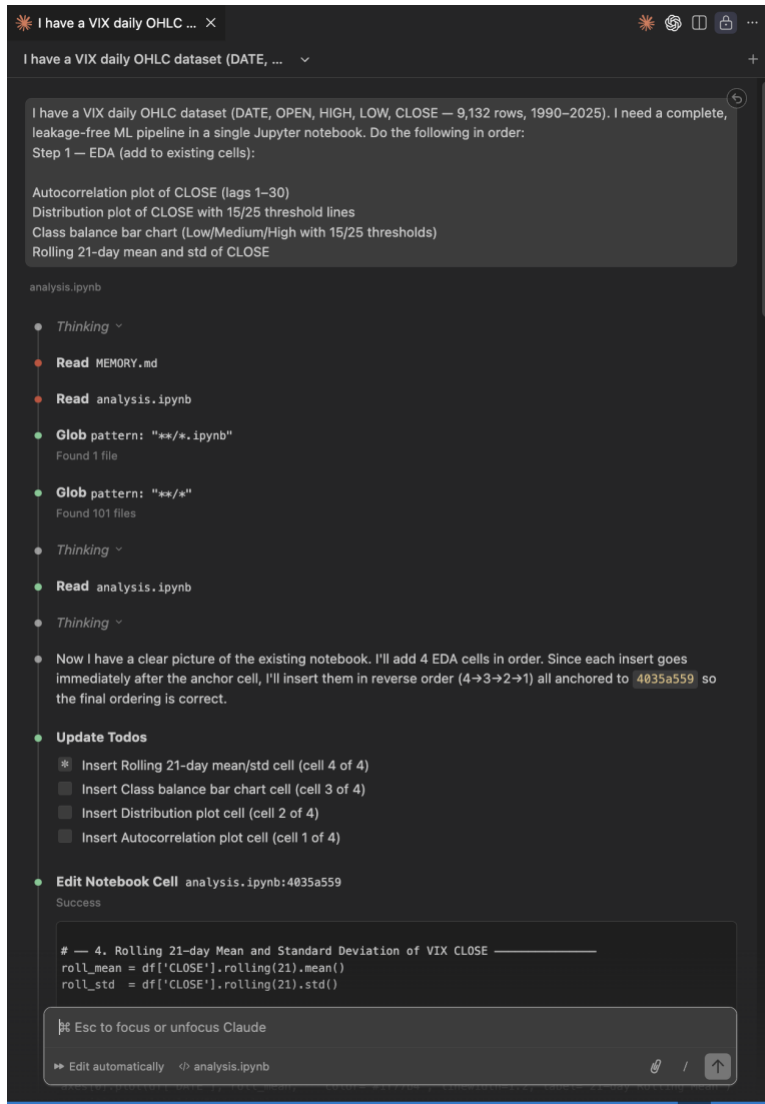


Figure A1: Agent task decomposition – planning EDA cell insertion order and reading notebook structure.

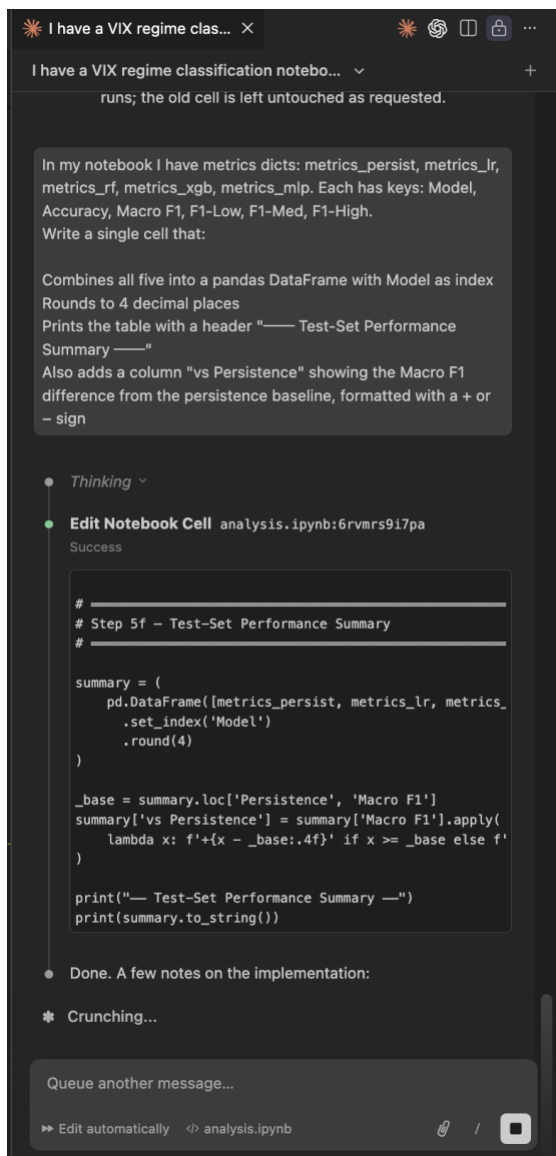


Figure A2: Agent code scaffolding – generating test-set performance summary cell with formatted output.

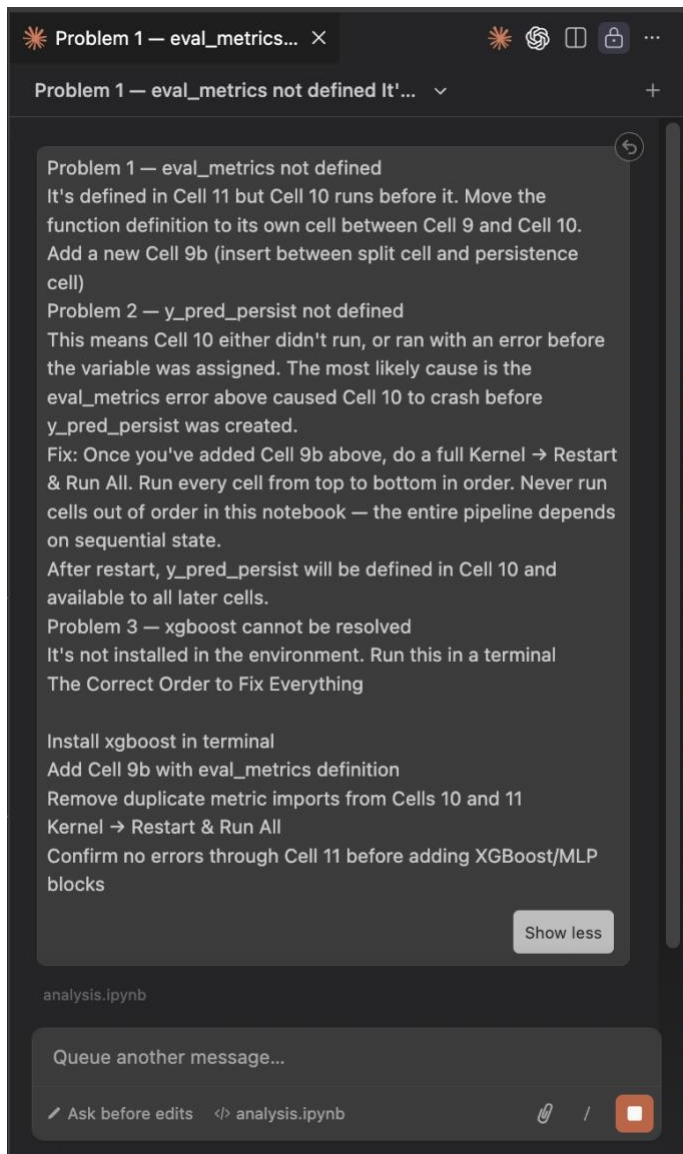


Figure A3: Agent error detection — identifying eval_metrics not defined runtime error and diagnosing root cause.

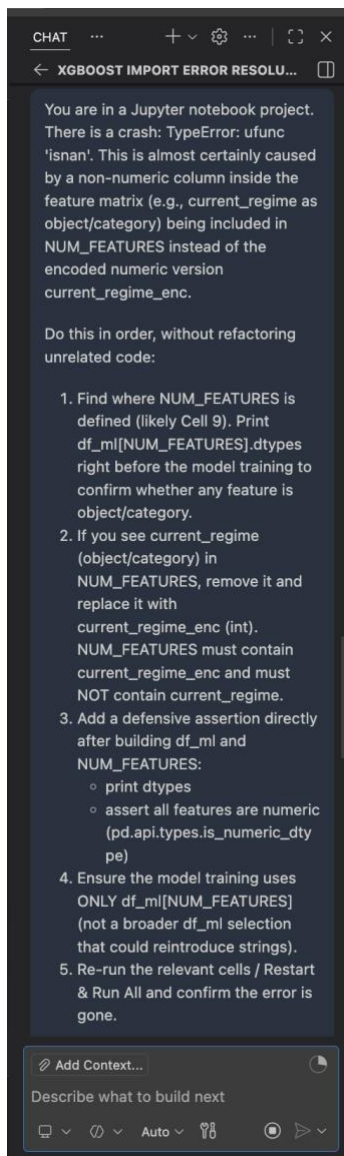


Figure A4: Agent error correction – diagnosing XGBoost import error with step-by-step resolution plan.

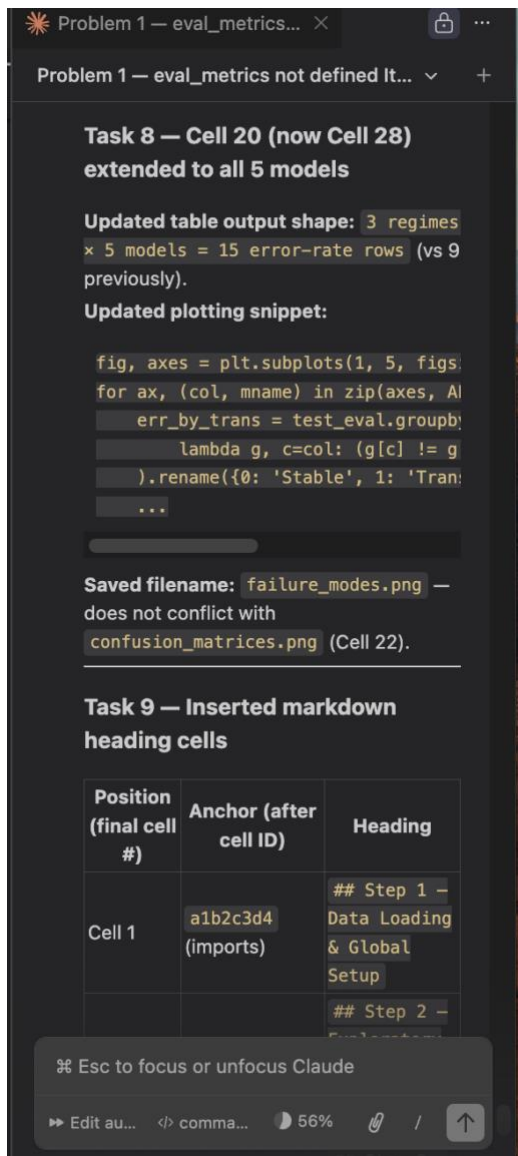


Figure A5: Agent verification – extending evaluation table to all 5 models and validating output dimensions.

Bibliography

- Cboe Global Markets (2020) "*The Cboe Volatility Index – VIX*." White paper.
- Breiman, L. (2001) "Random forests." *Machine Learning*, 45(1), 5–32.
- Garman, M. B., & Klass, M. J. (1980) "*On the estimation of security price volatilities from historical data*." *The Journal of Business*, 53(1), 67–78.
- Geron, A. (2017) *Hands-On Machine Learning with Scikit-Learn and TensorFlow*. O'Reilly.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009) *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.
- McNemar, Q. (1947) "*Note on the sampling error of the difference between correlated proportions or percentages*." *Psychometrika*, 12(2), 153–157.
- Whaley, R. E. (1993) "*Derivatives on market volatility: Hedging tools long overdue*." *The Journal of Derivatives*, 1(1), 71–84.